geneOptimize: An Object-Oriented Framework for Genetic Algorithms and Variable Selection in R

Version 1.1.0

Hui Zheng
huizheng1@gmail.com

February 26, 2026

Abstract

geneOptimize is an R package providing a modular, object-oriented framework for genetic algorithms (GAs) and memetic algorithms, built on the R6 class system. It supports binary and real-valued encodings, plug-in operator modules (selection, crossover, mutation), parallel fitness evaluation, and hybridization with local search. Version 1.1.0 introduces comprehensive input validation, a dedicated `maximize` parameter for minimization problems, explicit `lower`/`upper` bound arguments for real-valued optimization, selective memetic local-search controls (`local_search_every`, `local_search_top_k`), vectorized diversity tracking via `stats::dist()`, a corrected `MutationGaussian` interface, fully implemented `Population` and `GeneticAlgorithm` classes, and a comprehensive `testthat` test suite (60+ tests across six files).

Keywords: genetic algorithm, memetic algorithm, variable selection, evolutionary computation, R6, parallel computing.

Repository: `https://github.com/huizheng1-lab/geneOptimize`

Contents

1 Introduction **2**

2 Installation **2**

3 Architecture **2**
3.1 Class Hierarchy . . . 2

4 Functional Interface: `run_ga()` **3**
4.1 Signature . . . 3
4.2 Algorithm . . . 3
4.3 New in v1.1.0: Input Validation . . . 3
4.4 New in v1.1.0: Maximization and Minimization . . . 4
4.5 New in v1.1.0: Explicit Bounds for Real-Valued Optimization . . . 4

5 Operators **5**
5.1 Selection . . . 5
5.1.1 Tournament Selection . . . 5
5.1.2 Roulette-Wheel Selection (v1.1.0 fix) . . . 5
5.2 Crossover . . . 5
5.3 Mutation . . . 5

5.3.1 MutationSimple 5
5.3.2 MutationGaussian (signature fixed in v1.1.0) 5

6 Use Cases **6**
6.1 Binary Encoding: Variable Selection 6
6.2 Real-Valued Optimization: Ackley Function 7
6.3 Memetic Algorithms: Selective Local Search (v1.1.0) 7

7 Parallel Processing **8**

8 Population and GeneticAlgorithm Classes (v1.1.0) **8**
8.1 Population 8
8.2 GeneticAlgorithm 9

9 Visualization **9**
9.1 Diversity Tracking (v1.1.0 performance fix) 9
9.2 Configurable Convergence Threshold 10

10 Custom Operators **10**

11 Test Suite (v1.1.0) **10**

12 Summary of Changes in v1.1.0 **11**

13 Conclusion **11**

1 Introduction

Genetic algorithms are population-based meta-heuristics inspired by Darwinian natural selection [Holland, 1975]. At each generation a GA evaluates a population of candidate solutions (chromosomes), selects the fitter individuals, and applies genetic operators (crossover, mutation) to produce offspring. Over many generations the population converges towards high-quality solutions.

GAs are widely applied to problems that are non-convex, non-differentiable, or combinatorially complex — including feature selection for statistical models [Guyon & Elisseeff, 2003], hyperparameter optimization [Bergstra & Bengio, 2012], and real-valued function optimization [Goldberg, 1989].

geneOptimize provides a clean R6-based framework [Chang, 2021] that separates concerns into: *chromosome classes* (encoding and mutation logic), a *Population class* (initialization and evaluation), *operator classes* (modular, swappable selection, crossover, and mutation), a *functional interface* `run_ga()`, and a *plotting suite* for convergence diagnostics.

2 Installation

```r
# From CRAN
install.packages("geneOptimize")

# Development version
remotes::install_github("huizheng1-lab/geneOptimize")
```

3 Architecture

3.1 Class Hierarchy

geneOptimize is organized around an R6 class hierarchy employing both the *template method* and *strategy* patterns:

```
Chromosome (abstract base)
  +-- BinaryChromosome    bit-vector encoding, bit-flip mutation
  +-- RealChromosome      real-vector encoding, Gaussian noise
      mutation

Population                list of Chromosomes + evaluate() /
   get_best()
GeneticAlgorithm          orchestration wrapper (delegates to
   run_ga())

SelectionOperator (abstract)
  +-- SelectionTournament   tournament selection (configurable k)
  +-- SelectionRoulette     fitness-proportionate selection

CrossoverOperator (abstract)
  +-- CrossoverSinglePoint  single-point crossover
  +-- CrossoverUniform      uniform crossover (random mask)
  +-- CrossoverArithmetic   arithmetic blend (real-valued)

MutationOperator (abstract)
  +-- MutationSimple      bit-flip (binary) or scaled Gaussian (real)
  +-- MutationGaussian    Gaussian noise with configurable sigma field
```

Any operator can be replaced by a user-defined R6 class that inherits from the appropriate base class and implements the required method (`select()`, `mate()`, or `mutate()`).

4 Functional Interface: `run_ga()`

4.1 Signature

```
run_ga(
  fitness_fn,                                # Required
  n_genes,                                   # Required
  pop_size            = 50,
  generations         = 100,
  selection_fn        = SelectionTournament$new(),
  crossover_fn        = CrossoverSinglePoint$new(),
  mutation_fn         = MutationSimple$new(),
  crossover_rate      = 0.8,
  mutation_rate       = 0.01,
  type                = "binary",      # "binary" or "real"
  lower               = NULL,          # required when type = "real"
  upper               = NULL,          # required when type = "real"
  maximize            = TRUE,          # FALSE for minimization
  elitism_count       = 1,
  verbose             = TRUE,
  parallel            = FALSE,
  cores               = 1,
  local_search_fn     = NULL,
  local_search_every  = 1,
  local_search_top_k  = NULL,
  track_diversity     = FALSE,
  ...
)
```

4.2 Algorithm

1. **Initialize** population (binary: random 0/1; real: uniform over [`lower`, `upper`]).
2. **For each generation:**
   a. Evaluate fitness (optionally in parallel).
   b. Record `best_history`, `mean_history`, `worst_history`; optionally compute `diversity_history` via `stats::dist()`.
   c. Apply **elitism**: copy the top `elitism_count` individuals unchanged.
   d. **Selection** → build a mating pool.
   e. **Crossover** → generate offspring with probability `crossover_rate`.
   f. **Mutation** → perturb offspring gene-by-gene.
   g. **Local search** (optional): apply `local_search_fn` every `local_search_every` generations to all or the top-k individuals.
   h. Restore elites (override first rows of new population).
3. **Return** a `gene_opt_res` object.

4.3 New in v1.1.0: Input Validation

All parameters are validated at startup with informative `stop()` messages. Previously invalid inputs either silently produced wrong results or generated cryptic internal errors.

```
run_ga(42, n_genes = 5)
# Error: 'fitness_fn' must be a function.

run_ga(sum, n_genes = 5, type = "real")
# Error: 'lower' and 'upper' must be provided when type = "real".

run_ga(sum, n_genes = 5, crossover_rate = 1.5)
# Error: 'crossover_rate' must be in [0, 1].

run_ga(sum, n_genes = 5, type = "real", lower = 1, upper = 0)
# Error: All elements of 'lower' must be strictly less than ...
    'upper'.
```

Validated parameters: `fitness_fn`, `n_genes`, `pop_size` (≥ 2), `generations`, `crossover_rate` $\in [0, 1]$, `mutation_rate` $\in [0, 1]$, `type`, `elitism_count`, `maximize`, `local_search_every`, `local_search_top_k`, and the relationship `lower` $<$ `upper` element-wise.

4.4 New in v1.1.0: Maximization and Minimization

Prior to v1.1.0, `run_ga()` always maximized the fitness function; users had to negate their objective to minimize. Version 1.1.0 adds the `maximize` parameter:

```
# Minimize the Sphere function f(x) = sum(x^2); optimum at origin
result <- run_ga(
  fitness_fn   = function(x) sum(x^2),
  n_genes      = 5,
  type         = "real",
  lower        = rep(-5, 5),
  upper        = rep( 5, 5),
  maximize     = FALSE,           # NEW in v1.1.0
  pop_size     = 50,
  generations  = 100,
  verbose      = FALSE
)
```

4.5 New in v1.1.0: Explicit Bounds for Real-Valued Optimization

`lower` and `upper` are now explicit named parameters (previously they had to be threaded via `...`). A scalar is recycled to length `n_genes`:

```
result <- run_ga(
  fitness_fn = function(x) -sum((x - 2)^2),
  n_genes    = 3,
  type       = "real",
  lower      = c(-5, -5, -5),     # explicit, validated
  upper      = c( 5,  5,  5),
  verbose    = FALSE
)

# Scalar recycling
result <- run_ga(..., type = "real", lower = -10, upper = 10)
```

5 Operators

5.1 Selection

5.1.1 Tournament Selection

Draws k individuals at random and returns the one with the best fitness. Default $k = 3$.

```
sel <- SelectionTournament$new(k = 5)
```

5.1.2 Roulette-Wheel Selection (v1.1.0 fix)

Fitness-proportionate selection. The shift applied to negative fitness values previously used an undocumented magic constant of `0.01`; v1.1.0 replaces this with a documented value of 10^{-6} and explicitly handles the edge case of all-equal fitness values (previously undefined; now produces uniform random selection):

```
# v1.0.0 (opaque magic number, undefined equal-fitness behaviour)
adj_fitness <- if (min_fit < 0) fitness_values - min_fit + 0.01 else
    fitness_values

# v1.1.0 (documented, edge-case safe)
adj_fitness <- fitness_values - min_fit + 1e-6  # strictly positive
total  <- sum(adj_fitness)
probs  <- adj_fitness / total                    # uniform if all
    equal
```

5.2 Crossover

Class	Description
`CrossoverSinglePoint`	Swap gene segments at one random cut point
`CrossoverUniform`	Each gene independently inherited from either parent
`CrossoverArithmetic`	Weighted blend: $c_1 = \alpha p_1 + (1 - \alpha) p_2$

5.3 Mutation

5.3.1 MutationSimple

Handles both encodings:

- **Binary** — bit-flip with probability `mutation_rate`.
- **Real** — additive Gaussian noise $\mathcal{N}(0, (\texttt{upper} - \texttt{lower})/10)$, bounds-clipped.

5.3.2 MutationGaussian (signature fixed in v1.1.0)

The original `MutationGaussian` had an inconsistent method signature:

```
# v1.0.0 -- BROKEN: positional mismatch when called by run_ga()
mutate = function(genes, rate, sigma, lower, upper, ...)
# run_ga() called it as:  mutate(genes, type, mutation_rate, ...)
# so  rate   <- type          (wrong: character mapped to numeric
    arg)
#     sigma  <- mutation_rate (wrong: 0.01 used as standard
    deviation)
```

Version 1.1.0 aligns the signature with `MutationSimple` and makes `sigma` a class field:

```
# v1.1.0 -- correct and consistent
mut <- MutationGaussian$new(sigma = 0.05)

result <- run_ga(
  fitness_fn   = function(x) -sum(x^2),
  n_genes      = 4,
  type         = "real",
  lower        = rep(-1, 4),
  upper        = rep( 1, 4),
  mutation_fn = mut,          # plug in like any other operator
  pop_size     = 30,
  generations = 50,
  verbose      = FALSE
)
```

6 Use Cases

6.1 Binary Encoding: Variable Selection

A binary chromosome of length p encodes which of the p candidate features are included in a model. Fitness is the AUC of a logistic regression, penalized by the number of selected features to encourage sparsity:

```
library(geneOptimize)
set.seed(42)
n <- 200; p <- 15
X <- matrix(rnorm(n * p), n, p)
colnames(X) <- paste0("V", seq_len(p))

# Ground truth: only V1, V3, V5 affect the outcome
logits <- 1.5 * X[, 1] - 1.2 * X[, 3] + 0.8 * X[, 5]
y <- as.factor(ifelse(runif(n) < plogis(logits), "Class1", "Class0"))
df <- data.frame(y = y, X)

fitness_sparse_auc <- function(genes, penalty = 0.01) {
  if (sum(genes) == 0) return(0)
  vars  <- colnames(X)[genes == 1]
  model <- glm(as.formula(paste("y ~", paste(vars, collapse = " +
     "))),
               data = df, family = binomial)
  preds   <- predict(model, type = "response")
  roc_obj <- pROC::roc(y, preds, quiet = TRUE)
  as.numeric(pROC::auc(roc_obj)) - penalty * sum(genes)
}

result <- run_ga(
  fitness_fn   = fitness_sparse_auc,
  n_genes      = p,
  type         = "binary",
  pop_size     = 50,
  generations = 40,
  verbose      = FALSE
)
print(colnames(X)[result$best_chromosome == 1])
```

6.2 Real-Valued Optimization: Ackley Function

The Ackley function [Ackley, 1987] is a standard multi-modal benchmark with a global minimum of 0 at the origin:

$$f(\mathbf{x}) = -20\exp\left(-0.2\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}\right) - \exp\left(\frac{1}{d}\sum_{i=1}^{d}\cos(2\pi x_i)\right) + 20 + e \tag{1}$$

```
ackley <- function(x) {
  d <- length(x); a <- 20; b <- 0.2; cc <- 2 * pi
  -(- a * exp(-b * sqrt(sum(x^2) / d))
    - exp(sum(cos(cc * x)) / d)
    + a + exp(1))
}

result <- run_ga(
  fitness_fn    = ackley,
  n_genes       = 2,
  type          = "real",
  lower         = rep(-5, 2),
  upper         = rep( 5, 2),
  maximize      = FALSE,
  mutation_fn   = MutationGaussian$new(sigma = 0.1),
  crossover_fn  = CrossoverArithmetic$new(alpha = 0.5),
  pop_size      = 50,
  generations   = 100,
  verbose       = FALSE
)
```

6.3 Memetic Algorithms: Selective Local Search (v1.1.0)

Memetic algorithms [Moscato, 1989] hybridize evolutionary search with local refinement. Version 1.1.0 adds fine-grained controls:

Parameter	Type	Description
`local_search_fn`	function	Takes a chromosome, returns improved chromosome
`local_search_every`	integer	Apply every n generations (default: 1)
`local_search_top_k`	integer / NULL	Apply only to top-k individuals

```
# Hill-climbing local search
local_search <- function(chromosome) {
  candidate <- chromosome + rnorm(length(chromosome), 0, 0.01)
  if (sum(candidate^2) < sum(chromosome^2)) candidate else chromosome
}

result <- run_ga(
  fitness_fn          = function(x) -sum((x - 0.5)^2),
  n_genes             = 5,
  type                = "real",
  lower               = rep(0, 5),
  upper               = rep(1, 5),
  pop_size            = 30,
  generations         = 50,
```

```
  local_search_fn     = local_search,
  local_search_every = 5,        # apply every 5 generations
  local_search_top_k = 5,        # apply only to the 5 best
     individuals
  verbose             = FALSE
)
```

Prior to v1.1.0, local search was applied to *every* individual *every* generation, which was prohibitively expensive for costly local searches. The new parameters reduce the number of calls from `pop_size`×`generations` to at most `local_search_top_k`×⌈`generations`/`local_search_every`⌉.

7 Parallel Processing

`run_ga()` parallelizes fitness evaluation across multiple CPU cores. On Unix-like systems `parallel::mclapply()` is used; on Windows a socket cluster via `parallel::makeCluster()` is created and cleaned up automatically via `on.exit()`.

Requirements for `parallel = TRUE`:

- `fitness_fn` must be a *pure function*: no global-state reads or writes, no side effects (file I/O, database writes, etc.).
- All objects needed by `fitness_fn` must be captured in its closure — worker processes do not share the parent session's environment.
- Functions that depend on a shared random stream will behave non-deterministically across workers.

```
result <- run_ga(
  fitness_fn  = function(x) sum(x),
  n_genes     = 20,
  pop_size    = 100,
  generations = 200,
  parallel    = TRUE,
  cores       = 4,
  verbose     = FALSE
)
```

8 Population and GeneticAlgorithm Classes (v1.1.0)

Version 1.1.0 completes the OOP infrastructure that was partially stubbed in v1.0.0.

8.1 Population

```
# Binary population
pop <- Population$new(size = 20, n_genes = 10, type = "binary")
pop$evaluate(function(x) sum(x))
best <- pop$get_best()
cat("Best fitness:", best$fitness, "\n")

# Real-valued population
pop_real <- Population$new(size = 20, n_genes = 5, type = "real",
                           lower = -1, upper = 1)
```

The previously empty `evaluate()` and `get_best()` method bodies are now fully implemented. `get_best()` uses `vapply()` for type-safe fitness extraction.

8.2 GeneticAlgorithm

```
pop    <- Population$new(size = 30, n_genes = 10, type = "binary")
ga_obj <- GeneticAlgorithm$new(
  pop           = pop,
  fitness_fn    = function(x) sum(x),
  generations   = 50,
  crossover_rate = 0.8,
  mutation_rate  = 0.01
)
result <- ga_obj$run(verbose = FALSE)
plot_convergence(result)
```

`GeneticAlgorithm$run()` now delegates to `run_ga()`, ensuring that the object-oriented and functional interfaces remain consistent.

9 Visualization

Function	Description
`plot_convergence(x)`	Best fitness over generations
`plot_fitness_stats(x)`	Best, mean, and worst fitness with shaded band
`plot_diversity(x)`	Mean pairwise population diversity
`plot_diagnostics(x, conv_threshold)`	4-panel diagnostic (convergence, improvement, rate of change, summary)
`plot_compare(...)`	Overlay convergence curves from multiple runs

9.1 Diversity Tracking (v1.1.0 performance fix)

Diversity is computed as mean pairwise distance: Hamming (binary) or Euclidean (real). The v1.0.0 implementation used `Vectorize(outer())`, an R-level loop with $O(n^2)$ overhead. Version 1.1.0 replaces this with `stats::dist()`, a vectorized C-level computation:

```
# v1.0.0 -- slow R-level outer loop
hamming <- Vectorize(function(a, b) sum(a != b))
div_mat <- outer(1:pop_size, 1:pop_size,
                 function(i, j) hamming(pop[i,], pop[j,]))

# v1.1.0 -- vectorized C-level (stats::dist)
# Binary: Manhattan distance on 0/1 matrix == Hamming distance
div_mat <- as.matrix(stats::dist(population, method = "manhattan"))
diversity_history[gen] <- mean(div_mat[lower.tri(div_mat)])
```

Note: diversity tracking is disabled by default (`track_diversity = FALSE`) because even the improved implementation is $O(n^2)$ in population size. The error raised by `plot_diversity()` when diversity was not tracked now includes the corrective instruction:

```
# v1.0.0 -- unhelpful message
# Error: Diversity history not available. Enable diversity tracking
    in run_ga().

# v1.1.0 -- actionable message
```

```
# Error: Diversity history not available. Re-run run_ga() with
#   'track_diversity = TRUE':
#     run_ga(..., track_diversity = TRUE)
```

9.2 Configurable Convergence Threshold

`plot_diagnostics()` previously hard-coded the convergence check threshold at 0.001 (0.1% improvement over the last 10 generations). Version 1.1.0 exposes this as `conv_threshold`:

```
plot_diagnostics(result, conv_threshold = 0.005)  # 0.5% threshold
```

10 Custom Operators

Users can implement custom operators by inheriting from the appropriate base class:

```
library(R6)

# Epsilon-greedy selection: exploit the best with probability
   (1-eps),
# explore uniformly at random with probability eps.
EpsilonGreedy <- R6Class(
  "EpsilonGreedy",
  inherit = SelectionOperator,
  public = list(
    eps = 0.1,
    initialize = function(eps = 0.1) { self$eps <- eps },
    select = function(population, fitness_values, n = 1, ...) {
      if (runif(1) < self$eps) {
        return(population[sample(nrow(population), 1), ])
      }
      population[which.max(fitness_values), ]
    }
  )
)

result <- run_ga(
  fitness_fn   = function(x) sum(x),
  n_genes      = 15,
  selection_fn = EpsilonGreedy$new(eps = 0.2),
  pop_size     = 40,
  generations  = 60,
  verbose      = FALSE
)
```

11 Test Suite (v1.1.0)

Version 1.1.0 introduces a full `testthat` [Wickham, 2011] test suite. Prior to this release, `testthat` was listed in `Suggests` but no tests existed, creating an unacceptable regression risk.

File	Coverage
`test-chromosome.R`	`BinaryChromosome`, `RealChromosome`: initialization, mutation, deep clone
`test-operators.R`	All selection, crossover, and mutation operators including edge cases
`test-run_ga.R`	Input validation (12 tests), binary/real types, `maximize`, elitism monotonicity, diversity, local-search call counts
`test-methods.R`	`new_gene_opt_res`, `print`, `summary`, `plot`
`test-plotting.R`	All plotting functions, error conditions, actionable messages
`test-population.R`	`Population` initialization (binary and real), `evaluate()`, `get_best()`

Run the suite with:

```
devtools::test()            # or
testthat::test_package("geneOptimize")
```

12 Summary of Changes in v1.1.0

Component	Change
`run_ga()`	Added `maximize`, `lower`, `upper` as explicit named parameters; comprehensive input validation; added `local_search_every` and `local_search_top_k`; diversity tracking switched to `stats::dist()`; parallel requirements documented.
`MutationGaussian`	Fixed method signature to match `MutationSimple` interface; `sigma` is now a class field.
`SelectionRoulette`	Replaced magic offset 0.01 with documented 10^{-6}; handles all-equal fitness (uniform selection).
`plot_diagnostics()`	Added user-configurable `conv_threshold`; fixed rate-of-change denominator to `abs()`.
`plot_diversity()`	Error message includes corrective `run_ga()` call.
`Population`	Completed `initialize()`, `evaluate()`, `get_best()`.
`GeneticAlgorithm`	Completed `run()` (delegates to `run_ga()`).
Tests	60+ tests across 6 files; `testthat` suite fully implemented.
`DESCRIPTION`	Version bumped to 1.1.0; `grDevices` added to Imports; `pROC` added to Suggests.

13 Conclusion

geneOptimize provides a production-ready, extensible framework for genetic and memetic algorithms in R. Its R6-based architecture enables easy operator swapping and extension, while the `run_ga()` functional interface keeps common use cases concise. Version 1.1.0 focuses on correctness, usability, and testability — making the package suitable for both research prototyping and production variable selection pipelines.

References

Ackley, D. H. (1987). *A Connectionist Machine for Genetic Hillclimbing*. Kluwer Academic Publishers, Boston.

Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.

Chang, W. (2021). *R6: Encapsulated Classes with Reference Semantics*. R package version 2.5.1. https://CRAN.R-project.org/package=R6.

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, Reading, MA.

Guyon, I. and Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor.

Moscato, P. (1989). On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms. Technical Report 826, California Institute of Technology.

Wickham, H. (2011). testthat: Get started with testing. *The R Journal*, 3(1):5–10.